

pybind11 使用指南

面向有 C++ 基础、希望把 Python 接入 YaoEngine 的开发者

C++ <-> PYTHON

从第一个模块，到引擎脚本

Windows x64 / Premake 5 / Visual Studio 2026

版本依据：项目内 pybind11 3.1.0 源码与官方文档

本手册提供可复制的 C++、Python 与 Premake 示例。先通过独立示例验证编译和调用，再把相同机制接到引擎的脚本层。示例使用普通、带 GIL 的 CPython；不涉及自由线程 Python 或子解释器。

先选一种调用方向

目标	程序从哪里开始	对应入口
Python 使用 C++	启动 python.exe, import 编译好的 .pyd	PYBIND11_MODULE 第 4-7 页
C++ 使用 Python	启动引擎或 .exe, 在进程中初始化解释器	scoped_interpreter 第 8-9 页

阅读路线

先做什么	阅读页码
确认 SDK、路径与现有配置	02-03
构建模块，绑定函数、类与容器	04-07
嵌入脚本，处理生命周期与线程	08-11
批量数据、引擎接入、排错与参考	12-15

示例约定：CPython 3.12 x64，链接库名 python312。其他 Python 小版本请同步替换解释器、头文件、导入库和运行时。整理日期：2026-09-12。

环境准备与项目现状

把“头文件可见”“链接成功”“运行时可加载”分开确认。

pybind11 本身只有头文件，但 Python 绑定仍依赖 CPython 开发文件。安装完整 Python 开发环境，确认它与 C++ 工程均为 x64；不要只准备 python.exe 或单个 DLL。

内容	CPython 3.12 常见位置	用途
pybind11 头文件	Dep/pybind11/include	编译 C++ 绑定代码
Python 头文件	Python 根目录/include/Python.h	提供 Python C API 声明
Python 导入库	Python 根目录/libs/python312.lib	Windows 链接阶段使用
Python 运行时	python312.dll、Lib、DLLs 等	解释器与标准库运行

用你准备运行的解释器检查路径

POWERSHELL

```
# PowerShell: replace this with your own installation.
$py = "C:/Python312/python.exe"
& $py -c "import sys; print(sys.version); print(sys.base_prefix)"
& $py -c "import struct; print(struct.calcsize('P') * 8)"
& $py -c "import sysconfig; print(sysconfig.get_path('include'))"
```

以上 C:/Python312 是示例路径，不代表本机已安装在此处。venv 使用的 SDK 往往属于基础解释器，优先核对 sys.base_prefix，并确认 include 与 libs 实际存在。

当前 YaoEngine 已经完成的部分

EXISTING CONFIGURATION

```
-- Main premake5.lua includes:
include "YaoEngine-core/Dep/pybind11/pybind11.lua"

-- YaoEngine-core/core.lua contains:
includedirs { "Dep/pybind11/include" }
dependson { "pybind11" }
```

还需要补的配置

当前 pybind11 项目是 Utility，仅用于收录头文件与组织依赖。现有 core.lua 尚未提供 CPython 的 include、libs 和 Python 链接库；dependson 不会自动传递这些设置。

依据：[1] [9]；本地 core.lua、pybind11.lua、detail/common.h。编号链接见第 15 页。

一份独立示例的 Premake

建议在仓库根目录新建 PybindDemo 文件夹；本页与第 4、8 页的项目块写进同一份文件。

相对路径以 PybindDemo/premake5.lua 所在目录为基准。独立示例暂不链接 YaoEngine-core，可先确认 Python 工具链正常。这里只启用 Release，并保留调试符号。

PybindDemo/premake5.lua - shared settings

```
newoption { trigger = "python-home", value = "PATH",
  description = "Full CPython installation directory" }
newoption { trigger = "python-lib", value = "NAME",
  description = "Python import library, e.g. python312" }

local pyhome = _OPTIONS["python-home"]
local pylib = _OPTIONS["python-lib"]
assert(pyhome and pylib, "Set --python-home and --python-lib")
pyhome = path.getabsolute(pyhome)

workspace "PybindDemo"
  architecture "x64"
  configurations { "Release" }
  location "build"
  language "C++"
  cppdialect "C++17"
  staticruntime "Off"
  runtime "Release"
  buildoptions { "/utf-8" }
  symbols "On"
  optimize "On"
  targetdir "bin/{cfg.buildcfg}"
  objdir "obj/{prj.name}/{cfg.buildcfg}"
  includedirs {
    "../YaoEngine-core/Dep/pybind11/include",
    path.join(pyhome, "include"),
  }
  libdirs { path.join(pyhome, "libs") }
  links { pylib }
```

示例范围

这份配置面向 Windows 普通 CPython。staticruntime Off + runtime Release 使用 /MD；它是独立示例的选择。现有引擎使用 /MT，接入真实引擎时需统一相关自有 C++ 工程的运行库，见第 13 页。

依据：[10] [11] [12]。编号链接见第 15 页。

让 Python 调用 C++

一个 .pyd 就是 Python 可以 import 的原生扩展模块。

在共享配置末尾追加项目

```
PybindDemo/premake5.lua - append

project "yao_math"
    kind "SharedLib"
    targetprefix ""
    targetextension ".pyd"
    files { "bindings.cpp" }
```

实现并导出一个函数

```
PybindDemo/bindings.cpp

#include <pybind11/pybind11.h>
namespace py = pybind11;

float advance(float x, float speed, float dt) {
    return x + speed * dt;
}

PYBIND11_MODULE(yao_math, m) {
    m.doc() = "Small math helpers for YaoEngine";
    m.def("advance", &advance,
        py::arg("x"), py::arg("speed"),
        py::arg("dt") = 1.0f / 60.0f);
}
```

PYBIND11_MODULE 的第一个参数是模块名标识符，不加引号；第二个参数 m 表示当前模块。m.def 注册函数，py::arg 声明 Python 参数名与默认值。

三个名字必须一致

源码中的 yao_math、输出文件 yao_math.pyd、Python 语句 import yao_math 必须对应。模块名改变后应重新编译，不要只重命名旧 .pyd。

准备一条最小验证

```
PYTHON

import yao_math

x = yao_math.advance(x=2.0, speed=4.0, dt=0.5)
assert abs(x - 4.0) < 1e-6
print(x) # 4.0
help(yao_math.advance)
```

依据：[1] [10]。编号链接见第 15 页。

生成、编译与验证

以下命令从 YaoEngine 仓库根目录执行；先修改 Python 安装位置。

1. 准备目录与源码

DIRECTORY LAYOUT

```
YaoEngine-DEV/
  Script/premake5.exe
  YaoEngine-core/Dep/pybind11/include/
  PybindDemo/
    premake5.lua
    bindings.cpp
    main.cpp           # used on page 8
    scripts/movement.py # used on page 8
```

2. 生成 VS 2026 解决方案

POWERSHELL

```
$pyRoot = "C:/Python312"
$py = Join-Path $pyRoot "python.exe"
& ./Script/premake5.exe `
  --file=PybindDemo/premake5.lua `
  "--python-home=$pyRoot" --python-lib=python312 vs2026
```

打开 PybindDemo/build/PybindDemo.slnx，选择 Release，然后生成 yao_math。输出应为 PybindDemo/bin/Release/yao_math.pyd。工程文件由 VS 2026 打开。

3. 用相同 Python 导入模块

POWERSHELL

```
Push-Location ./PybindDemo/bin/Release
& $py -c "import yao_math; print(yao_math.advance(2, 4, 0.5))"
Pop-Location
# Expected output: 4.0
```

成功标准

Premake 显示 Done，解决方案中出现 yao_math；VS 成功生成 .pyd；用相同解释器 import 后返回 4.0。生成、编译、运行这三步都成功，才算完成最小验证。

本地练习可使用简单 .pyd 后缀。发布多个 Python 版本的二进制时，应使用目标解释器的 EXT_SUFFIX 并分别构建；不要假定不同小版本之间可以直接共用。

依据：[1] [9] [10]。编号链接见第 15 页。

绑定类、字段与方法

本页是 bindings.cpp 的完整替换版本；替换后重新生成 yao_math。

PybindDemo/bindings.cpp - replace

```
#include <pybind11/pybind11.h>
namespace py = pybind11;

struct Vec2 {
    float x, y;
    Vec2(float px, float py) : x(px), y(py) {}
    void translate(float dx, float dy) {
        x += dx;
        y += dy;
    }
    float length_squared() const { return x*x + y*y; }
};

PYBIND11_MODULE(yao_math, m) {
    py::class_<Vec2, py::smart_holder>(m, "Vec2")
        .def(py::init<float, float>()),
        py::arg("x") = 0.0f, py::arg("y") = 0.0f)
        .def_readwrite("x", &Vec2::x)
        .def_readwrite("y", &Vec2::y)
        .def("translate", &Vec2::translate,
            py::arg("dx"), py::arg("dy"))
        .def_property_readonly("length_squared",
            &Vec2::length_squared);
}
```

PYTHON

```
import yao_math
v = yao_math.Vec2(3, 4)
assert v.length_squared == 25
v.x = 6
v.translate(dx=1, dy=2)
assert (v.x, v.y) == (7, 6)
```

接口	含义
py::init<...>()	绑定构造函数
def / def_static	绑定实例方法 / 静态方法
def_readwrite / def_readonly	直接暴露可写 / 只读字段
def_property / def_property_readonly	通过 getter、setter 暴露属性

pybind11 v3 推荐在多数场景使用 py::smart_holder。它帮助处理智能指针的所有权转换，但不能修复已失效的引擎对象或悬空指针。

依据：[2] [3]。编号链接见第 15 页。

类型转换与函数接口

从简单标量开始；容器与回调需要额外头文件。

C++ 类型	Python 表现	需包含的头文件
int / float / bool	int / float / bool	pybind11/pybind11.h
std::string	通常是 str；二进制数据用 bytes	pybind11/pybind11.h
std::vector<T>	list	pybind11/stl.h
std::map<K,V>	dict	pybind11/stl.h
std::optional<T>	值或 None (C++17)	pybind11/stl.h
std::function<...>	可调用对象	pybind11/functional.h

列表输入与返回值

C++ - add header at top, binding inside module

```
#include <pybind11/stl.h>
#include <vector>

// Add inside the existing PYBIND11_MODULE block:
m.def("scaled", [](std::vector<float> values, float s) {
    for (auto &v : values) v *= s;
    return values;
}, py::arg("values"), py::arg("s"));
```

PYTHON

```
values = [1.0, 2.0, 3.0]
assert yao_math.scaled(values, 2.0) == [2.0, 4.0, 6.0]
assert values == [1.0, 2.0, 3.0]
```

STL 自动转换通常会复制数据

即使 C++ 参数写成 `std::vector<T>&`，通过 `stl.h` 接收 Python list 时也不等于共享同一容器。需要大批量数据或就地修改时，优先考虑 NumPy / buffer，见第 12 页。

参数与重载

重载函数可用 `py::overload_cast<float>(&sample)` 等方式选定签名。调用参数不匹配会产生 `TypeError`；接口设计上优先采用清晰名称与少量重载，减少隐式转换带来的歧义。

依据：[4] [5]；本地 docs/advanced/cast/functional.rst。编号链接见第 15 页。

让 C++ 调用 Python

保留第 3 页的共享配置，新增一个 ConsoleApp 项目。

PybindDemo/premake5.lua - append

```
project "py_runner"
    kind "ConsoleApp"
    files { "main.cpp" }
    debugdir (path.getabsolute("."))
    debugenvs { "PATH=" .. pyhome .. " ;%PATH%" }
```

PybindDemo/main.cpp

```
#include <pybind11/embed.h>
#include <iostream>
namespace py = pybind11;

int main() {
    py::scoped_interpreter python{};
    try {
        auto sys = py::module_::import("sys");
        sys.attr("path").attr("insert")(0, "scripts");
        auto script = py::module_::import("movement");
        float x = script.attr("update")(2.0f, 0.5f)
            .cast<float>();
        std::cout << "x = " << x << '\n';
    } catch (const py::error_already_set &e) {
        std::cerr << e.what() << '\n';
        return 1;
    }
}
```

PybindDemo/scripts/movement.py

```
def update(x, dt):
    return x + 4.0 * dt
```

解释器 guard 在 try 外创建，因此捕获和格式化 Python 异常时解释器仍然存活。先析构模块、返回值和异常，再离开 main 关闭解释器。

构建后从 PybindDemo 目录运行

POWERSHELL

```
$pyRoot = "C:/Python312"
$env:PATH = "$pyRoot;$env:PATH"
Push-Location ./PybindDemo
./bin/Release/py_runner.exe
Pop-Location
# Expected output: x = 4
```

scripts 使用相对路径，因此本示例明确固定工作目录。正式引擎应由项目配置或可执行文件位置确定脚本根目录，再把绝对路径加入 sys.path。

依据：[6] [8] [9]。编号链接见第 15 页。

让脚本操作 C++ 对象

独立引擎演示：嵌入模块 yao，脚本每帧修改 Actor。

PybindDemo/engine.cpp

```
#include <pybind11/embed.h>
#include <iostream>
namespace py = pybind11;

struct Actor {
    float x = 0.0f;
    void move(float dx) { x += dx; }
};

PYBIND11_EMBEDDED_MODULE(yao, m) {
    py::class_<Actor, py::smart_holder>(m, "Actor")
        .def(py::init<>())
        .def_readwrite("x", &Actor::x)
        .def("move", &Actor::move);
}

int main() {
    py::scoped_interpreter python{};
    try {
        auto sys = py::module_::import("sys");
        sys.attr("path").attr("insert")(0, "scripts");
        auto yao = py::module_::import("yao");
        auto actor = yao.attr("Actor")();
        auto update = py::module_::import("behavior")
            .attr("on_update");
        for (int frame = 0; frame < 3; ++frame)
            update(actor, 0.5f);
        std::cout << actor.attr("x").cast<float>()
            << '\n';
    } catch (const py::error_already_set &e) {
        std::cerr << e.what() << '\n';
        return 1;
    }
}
```

PybindDemo/scripts/behavior.py

```
import yao

def on_update(actor: yao.Actor, dt: float):
    actor.move(4.0 * dt)
```

复制第 8 页 py_runner 的项目块，将项目名改为 engine_demo、源文件改为 engine.cpp。重新生成并构建后，在 PybindDemo 目录运行 bin/Release/engine_demo.exe，应输出 6。

与真实引擎的区别

示例 Actor 是教学类型，由 Python 包装对象拥有，不是现有 ECS 实体。真实场景建议暴露实体 ID 与有效性检查，再由引擎查询和修改组件。不要直接假定 ECS 内部地址长期稳定。

依据：[3] [6]；Actor 与帧循环为本手册原创示例。编号链接见第 15 页。

对象所有权与生命周期

返回指针或引用之前，先回答：谁负责销毁？谁保证它还活着？

返回方式 / 策略	含义	适合场景
值 / copy	Python 得到独立值或副本	小型 Vec2、查询结果
move	将可移动结果交给 Python	返回临时结果
reference	Python 不拥有原对象	外部保证生命周期
reference_internal	借用内部对象，并维持父对象存活	返回父对象的成员引用
take_ownership	Python 最终负责销毁该对象	仅限确实移交所有权的对象

父对象拥有成员：明确写出返回策略

```
C++ - declarations outside module; binding inside

// Vec2 has already been bound in this module.
struct Body {
    Vec2 position{0.0f, 0.0f};
    Vec2 &get_position() { return position; }
};

// Add inside the existing module block:
py::class_<Body, py::smart_holder>(m, "Body")
    .def(py::init<>())
    .def("position", &Body::get_position,
        py::return_value_policy::reference_internal);
```

reference_internal 可以让返回的成员包装对象保持父对象存活；它不会阻止引擎在别处移除组件，也不能保证可扩容容器的元素地址不变。

引擎层的实用规则

1. 小型数学类型先按值传递，减少所有权问题。
2. 由场景或资源管理器持有的对象，不要误用 take_ownership。
3. 返回裸指针时显式声明策略；默认策略对某些裸指针会接管所有权。
4. 脚本长时间持有实体时，优先用 “ID + 代数/版本 + 有效性检查” 的句柄。

解释器也有生命周期

所有 py::object、py::function、模块缓存和持有 Python 引用的成员，都必须在解释器关闭前销毁。把它们作为成员时，先声明 interpreter guard，使 guard 最后析构；销毁 Python 引用时仍需正确持有 GIL。

依据：[3] [5] [6]；实体句柄为针对 YaoEngine 的设计建议。编号链接见第 15 页。

GIL 与工作线程

本页仅讨论普通、带 GIL 的 CPython；GIL 不替代引擎自身的数据同步。

从 Python 调用绑定函数时，默认已经持有 GIL。仅执行纯 C++ 计算的耗时区间可以释放 GIL；调用 Python、复制或销毁 Python 对象时，需要持有对应解释器的 GIL。[7]

C++ - binding fragment

```
// heavy_compute must not touch Python objects or callbacks.
m.def("heavy_compute", &heavy_compute,
      py::call_guard<py::gil_scoped_release>());
```

主线程等待工作线程时，先让出 GIL

C++ - standalone threading example

```
#include <pybind11/embed.h>
#include <iostream>
#include <thread>
namespace py = pybind11;

int main() {
    py::scoped_interpreter python{};
    {
        py::gil_scoped_release release;
        std::thread worker([] {
            py::gil_scoped_acquire acquire;
            try {
                auto math = py::module_::import("math");
                std::cout << math.attr("sqrt")(81).cast<int>();
            } catch (const py::error_already_set &e) {
                std::cerr << e.what();
            }
        });
        worker.join();
    } // Main thread reacquires the GIL here.
}
```

若主线程一直持有 GIL 并直接 join，而工作线程正在等待 GIL，两边就会互相等待。上面用 release 作用域包住线程创建和 join；线程内 acquire 比 Python 局部对象活得更久。

接入帧循环时

先以主线程统一执行脚本，确认功能后再引入并发。不要在已释放 GIL 的区间调用 Python 回调；不要把全局 py::object 当作普通 C++ 静态对象。对引擎场景、渲染上下文和资源队列仍需遵守各自线程规则。

依据：[6] [7]。编号链接见第 15 页。

批量数据与异常

高频接口应减少跨语言调用次数，并明确失败时的行为。

NumPy: 严格接收连续 float32 数组并就地修改

```
C++ - add header at top, binding inside module

#include <pybind11/numpy.h>

// Add inside the existing module block:
m.def("scale_inplace",
      [](py::array_t<float, py::array::c_style> a, float s) {
          auto data = a.mutable_unchecked<1>();
          for (py::ssize_t i = 0; i < data.shape(0); ++i)
              data(i) *= s;
      }, py::arg("a").noconvert(), py::arg("s"));
```

PYTHON - requires NumPy in the selected environment

```
import numpy as np
import yao_math

a = np.array([1, 2, 3], dtype=np.float32)
yao_math.scale_inplace(a, 2.0)
assert np.allclose(a, [2, 4, 6])
```

本例要求一维、可写、C 连续、float32 的数组；noconvert 拒绝用隐式转换产生替代数组。维数和可写性检查由相关接口完成，不兼容输入会报错。默认转换若产生副本，就不能保证修改回到原数组。[13]

C++ 抛出错误，让 Python 明确失败

```
C++ - add header at top, binding inside module

#include <stdexcept>

m.def("set_speed", [](float speed) {
    if (speed < 0.0f)
        throw std::invalid_argument("speed must be nonnegative");
    return speed;
});
```

std::invalid_argument 默认转为 Python ValueError。反向调用时，Python 抛出的异常通常表现为 C++ 的 py::error_already_set；在解释器仍存活且 GIL 有效的范围内记录 e.what()。[8]

给帧循环定义失败策略

脚本出错后，可以停用当前脚本并保留完整 traceback，避免每帧重复刷屏。批量更新优先传递数组或一组实体 ID，不要为每个坐标、每个实体分别跨语言调用。性能收益应由实际场景测量。

依据：[8] [13]；性能与错误恢复策略为工程建议。编号链接见第 15 页。

接回 YaoEngine 的位置

以下是下一步接入方案说明；本手册没有修改现有引擎源码。

层级	建议负责的事情
应用入口 / 编辑器	创建一次解释器；退出前停止脚本任务并清理 Python 引用
脚本系统	管理模块、on_start / on_update / on_stop 回调及 traceback
绑定层	导出精简引擎 API；检查实体有效性与线程要求
场景 / ECS	持有真实对象；以稳定句柄控制脚本可访问范围

构建配置应落在哪一层

编译含 pybind11 代码的项目需要两个 include 路径；生成最终 .exe 或 DLL 的链接项目需要 CPython 库。为了便于排错，在实际消费者上显式配置，并检查生成工程的链接依赖。

```
PREMAKE - adapt paths and Python version

-- Example only: apply to the actual consuming project.
local pythonRoot = "C:/Python312"
includedirs {
    "Dep/pybind11/include",      -- relative to core.lua
    path.join(pythonRoot, "include"),
}

-- Also configure the final executable / DLL as needed.
libdirs { path.join(pythonRoot, "libs") }
links { "python312" }
```

先处理现有 /MT 与示例 /MD 的差异

当前 YaoEngine-core 使用 staticruntime On。独立例子使用 Off。把示例直接链接到引擎可能引入运行库冲突；统一起链接的自有 C++ 工程配置，再做集成验证，不要靠忽略默认库掩盖 LNK2038 等错误。[11] [12]

嵌入模块应确保进入最终可执行文件

PYBIND11_EMBEDDED_MODULE 必须放在全局作用域，并且对应目标文件要被链接进 .exe。初期可把绑定 .cpp 直接列入应用项目。若放在静态库里，只有注册副作用的目标文件可能被丢弃；用实际引用的锚点函数等方式确保它被拉入。

部署时还要安排与 SDK 匹配的 Python DLL、标准库、扩展模块和脚本目录。设置 PATH 只解决部分 DLL 查找问题，不等于配置了完整 Python 模块搜索路径。

依据：[6] [9]；本地 core.lua；静态库保留方式为链接集成建议。编号链接见第 15 页。

常见报错速查

先确定问题发生在编译、链接、加载还是执行阶段，再检查对应配置。

现象	优先检查
找不到 pybind11/pybind11.h	include 路径应指向 pybind11/include，而非 include/pybind11。
找不到 Python.h	消费者未添加 CPython include；确认 SDK 目录含 Python.h。
LNK1104: python312.lib	确认 libs 路径与文件名，避免把 3.13 的安装和 python312 混用。
LNK2019: Py_* 未解析	检查最终链接项目是否有 Python 导入库，且平台都是 x64。
ModuleNotFoundError	确认 .pyd 或 .py 所在目录在 sys.path，并核对运行工作目录。
ImportError: DLL load failed	检查 Python DLL、扩展的其他 DLL 依赖、位数与 Python 小版本。
缺少 PyInit_yao_math	PYBIND11_MODULE 名称与 import 名称不符，或只是改了文件名。
LNK2038 / RuntimeLibrary	自有 C++ 目标的 /MD、/MT、Debug/Release 或其他 ABI 设置不一致。
退出时崩溃 / 随机引用错误	检查 Python 引用是否晚于解释器析构，以及裸指针和返回策略。
线程卡住	主线程是否持 GIL 等待工作线程？检查 join 与锁的持有顺序。
修改 vector 后 list 没变化	stl.h 转换通常产生副本；改为返回新列表或设计数组接口。

一条诊断命令

```
POWERSHELL - $py is your selected python.exe
& $py -c "import sys; print(sys.executable); print(sys.path)"
```

Debug 不等于 Debug Python

本仓库的 pybind11 在常见 MSVC Debug 场景会处理 Python 头文件对调试库的自动选择。不能简单认为选 Debug 就必须安装 Debug Python。若实际使用 Py_DEBUG 构建，则必须匹配其库、ABI 与扩展命名。入门先用本手册的 Release + symbols On。

依据：[1] [4] [5] [6] [7] [9]；本地 wrap_include_python_h.h。编号链接见第 15 页。

速记与参考资料

官方资料均为可点击链接；示例与说明已按当前仓库内容校核。

你想做什么	常用入口
导出函数 / 类	<code>m.def</code> / <code>py::class_</code> / <code>py::smart_holder</code>
导入并调用脚本	<code>py::module_::import</code> / <code>attr</code> / <code>cast<T></code>
管理解释器与线程	<code>scoped_interpreter</code> / <code>gil_scoped_acquire</code> / <code>gil_scoped_release</code>
查看 Python 错误	捕获 <code>py::error_already_set</code> ，记录 <code>e.what()</code>

官方文档

- [1] 基础：函数、命名与扩展模块 <https://pybind11.readthedocs.io/en/stable/basics.html>
- [2] 类、构造函数与属性 <https://pybind11.readthedocs.io/en/stable/classes.html>
- [3] 智能指针与 `smart_holder` https://pybind11.readthedocs.io/en/stable/advanced/smart_ptrs.html
- [4] STL 容器与复制语义 <https://pybind11.readthedocs.io/en/stable/advanced/cast/stl.html>
- [5] 返回策略与函数接口 <https://pybind11.readthedocs.io/en/stable/advanced/functions.html>
- [6] 嵌入解释器与生命周期 <https://pybind11.readthedocs.io/en/stable/advanced/embedding.html>
- [7] GIL 与线程注意事项 <https://pybind11.readthedocs.io/en/stable/advanced/misc.html>
- [8] 异常转换 <https://pybind11.readthedocs.io/en/stable/advanced/exceptions.html>
- [9] CPython: Windows 原生扩展构建 <https://docs.python.org/3/extending/windows.html>
- [10] Premake: `targetextension` <https://premake.github.io/docs/targetextension/>
- [11] Premake: `staticruntime` <https://premake.github.io/docs/staticruntime/>
- [12] Premake: `runtime` <https://premake.github.io/docs/runtime/>
- [13] NumPy 与 `buffer` 接口 <https://pybind11.readthedocs.io/en/stable/advanced/pycpp/numpy.html>

版本与验证边界

版本取自仓库 `YaoEngine-core/Dep/pybind11/include/pybind11/detail/common.h`: 3.1.0。该源码要求 Python 3.9 或更新版本。本手册具体构建示例采用 CPython 3.12 x64；本地已有 Premake 5.0.0-beta8。

已验证：三个完整示例的 C++ 源码已在 CPython 3.12.14 x64、MSVC Release 下构建并运行，分别输出 4.0、`x = 4`、6。随后新增 /utf-8 编码选项，已重新生成工程，未再次编译。其余进阶片段未逐项运行；尚未编译或集成整个 YaoEngine。